

TEAM 6 · SECURE SoC DESIGN PROJECT

Project **Cryptonite**

A fully encrypted RISC-V CPU with a hardware AES-128 accelerator

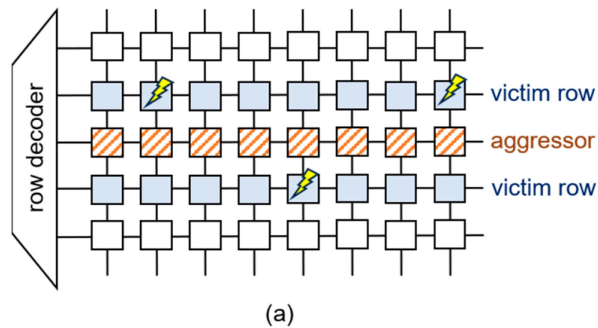
Lorenza Giovannetti · Orlan Forshaw · Morgan Pressman · Theo Banks

OVERVIEW

Dedicated Cryptography Hardware

- **Software isn't enough**
- Performance
- Security-Isolation
- Always-on Encryption
- Tamper Resistance

Rowhammer Attack



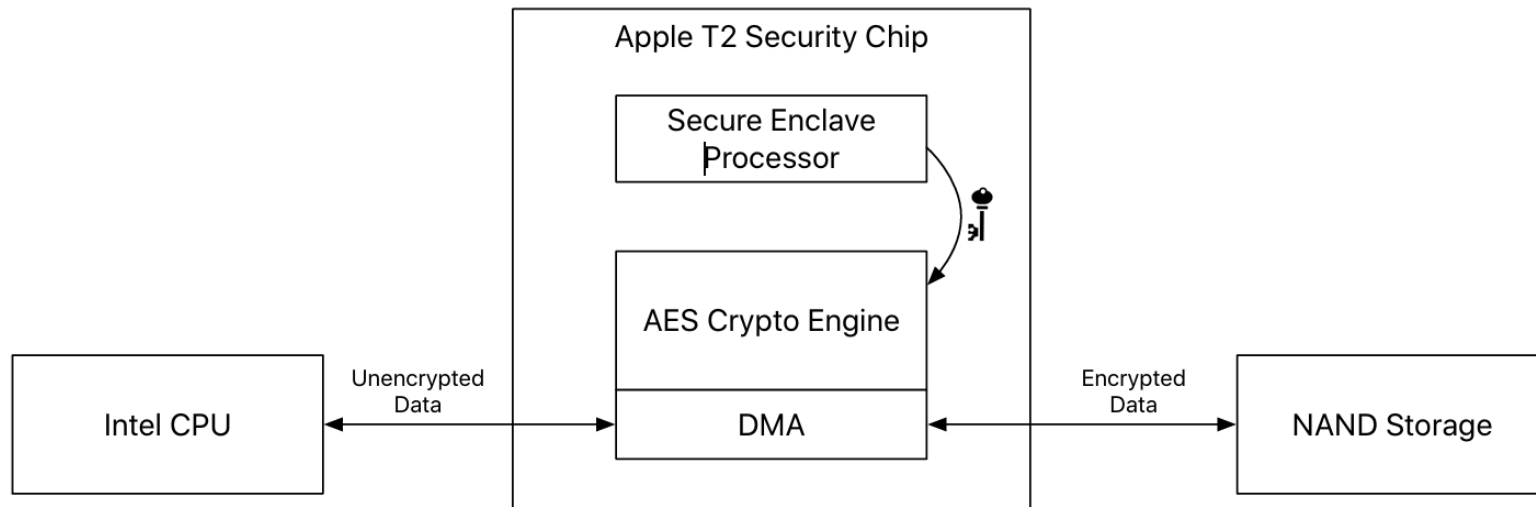
```
hammer:  
  mov (X), %eax // read from address X  
  mov (Y), %ebx // read from address Y  
  clflush (X)    // flush cache for address X  
  clflush (Y)    // flush cache for address Y  
  mfence  
  jmp hammer
```

(b)

OVERVIEW

Apple T2 security chip

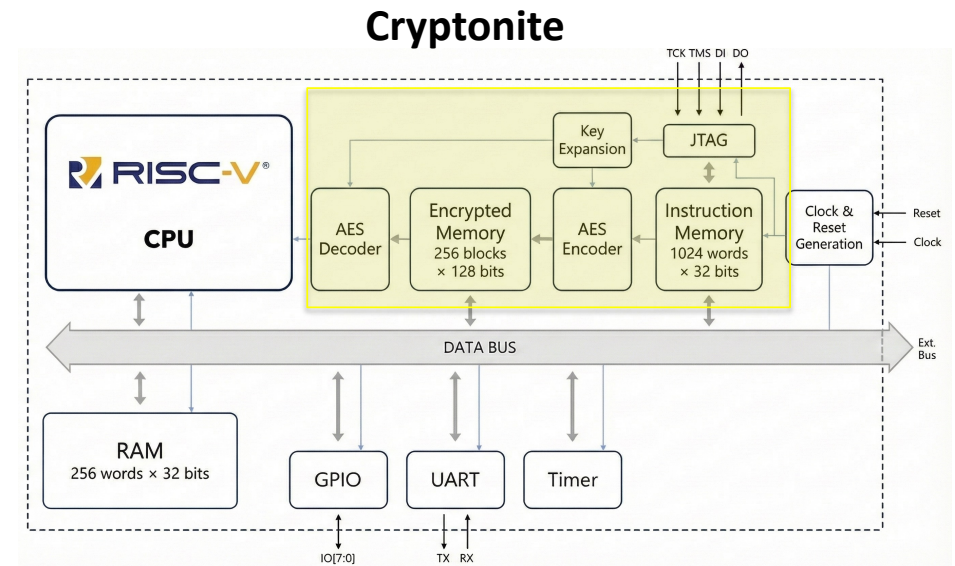
Internal Volume Encryption



OVERVIEW

Project Cryptonite

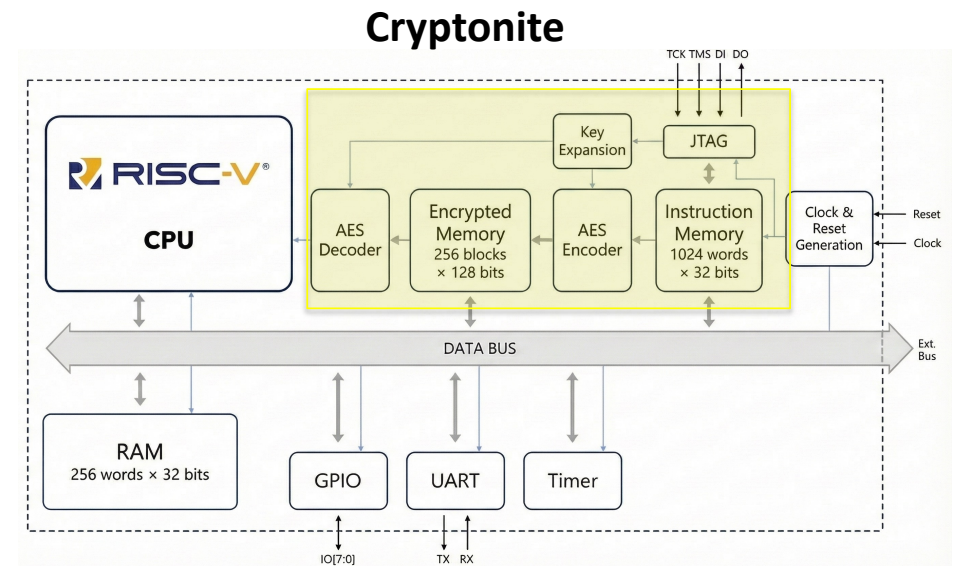
- **Custom AES-128 Cryptographic Accelerator**
- **Protect stored data**
- **User Input Key**
 - Encryption
 - Decryption + Running
- **Enhancing Luca's CPU design**



OVERVIEW

Project Cryptonite

- **Modular**
 - Development
 - Component isolation
- **Incremental stretch goals**
 - Encrypt both instruction and data memories
 - pipelining
 - Opaque to software



OVERVIEW

Why AES-128?

- **Brute-Force Resistance**
 - 128 bit key --> 32 undecillion combinations
- **Requires fewer processing rounds (9)**
 - AES-192 = (12) AES-256 = (14)
- **Hardware-friendly**

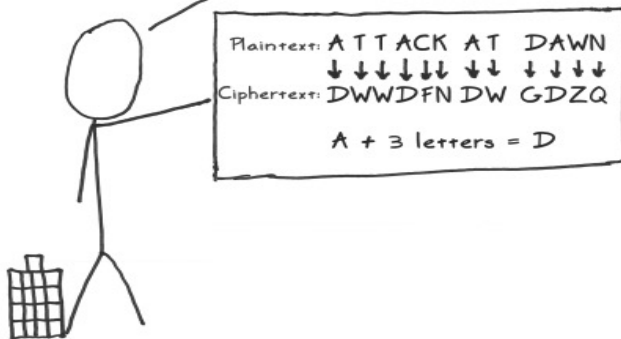
How does AES-128 work?

A symmetric block cipher that scrambles 128 bits of data into ciphertext through ten rounds of substitution and permutation

Claude Shannon in Bell Labs

Big Idea #1: Confusion

It's a good idea to obscure the relationship between your real message and your 'encrypted' message. An example of this 'confusion' is the trusty ol' Caesar Cipher:



Big Idea #2: Diffusion

It's also a good idea to spread out the message. An example of this 'diffusion' is a simple column transposition:



Four Operations

Every round of AES is built from the same four steps. Without the key, the ciphertext should be indistinguishable from random noise.

Confusion

Sub-bytes
Add Round Key

Diffusion

Shift Row
Mix Columns

Flip one input
bit...

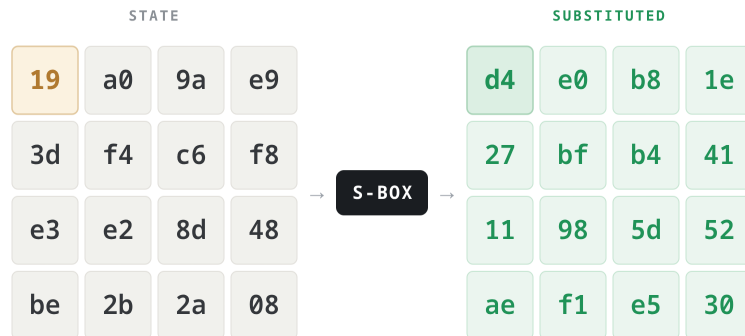


Aproximately half of the output bits change $\approx 64/128$

Confusion

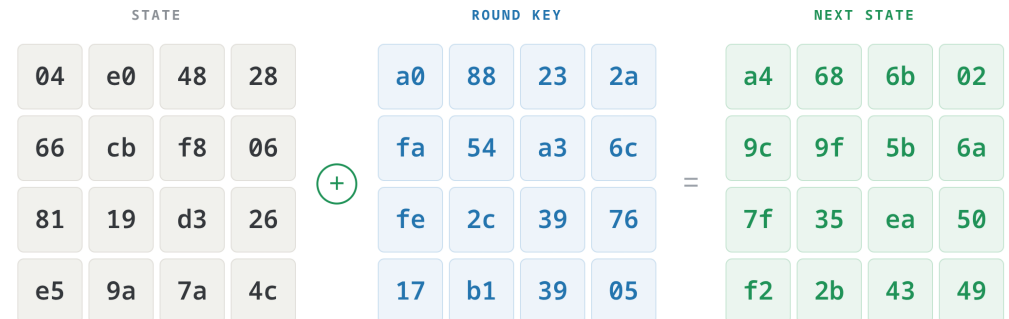
Sub-Bytes

- Resistant to linear and differential cryptanalysis
- Scrambles locally
- Multiplicative inverse in $GF(2^8)$, then affine map



AddRoundKey

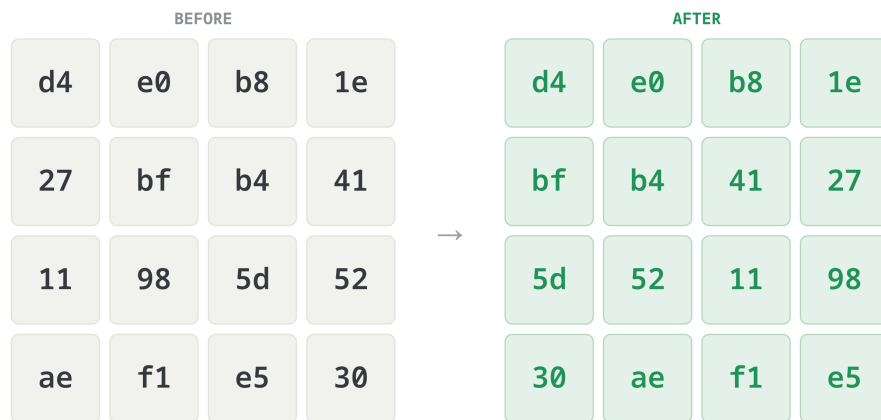
- Injects secret transformed key into state
- Obfuscates data by XOR-ing with a different round key each loop
- XOR is it's self-inverse, cheap for decryption



Diffusion

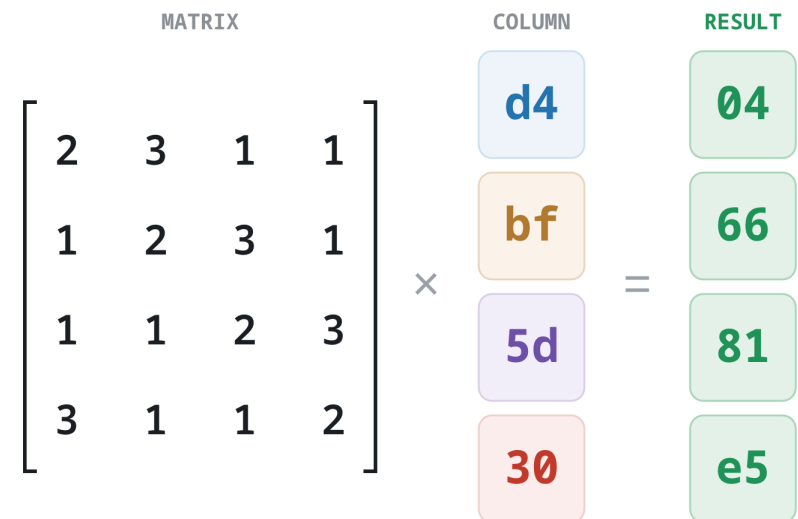
Shift Rows

- Each byte is rotated cyclically left by its row number
- Inter-column mixing



Mix Columns

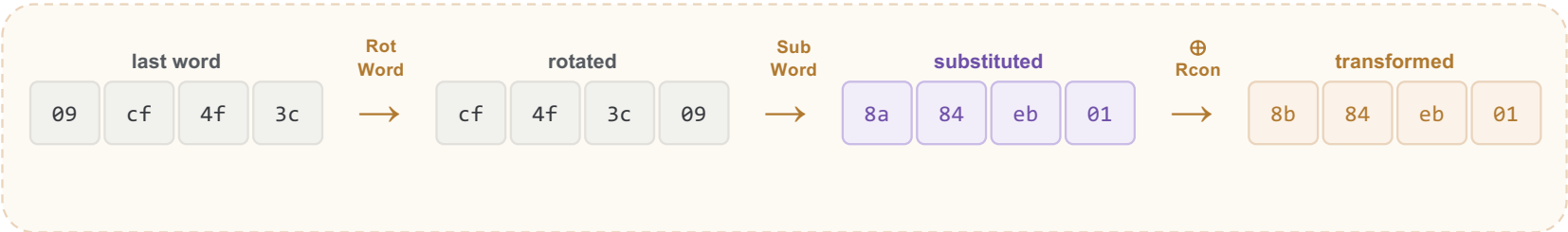
- State multiplied by matrix
- Inter-row mixing



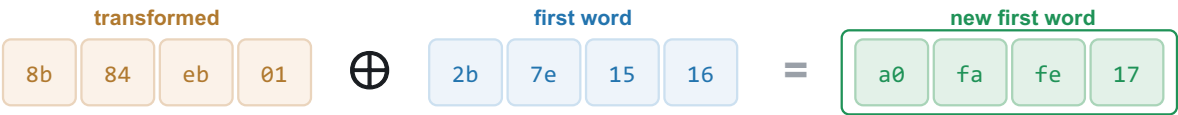
How the round keys are made

The key is four 4-byte words - each round needs its own, so one key grows into eleven.

STEP 1 · transform the last word



STEP 2 · XOR with the first word



Rcon is the round constant — one byte, different every round, so the eleven round keys stay unrelated.

01	02	04	08	10	20	40	80	1b	36
----	----	----	----	----	----	----	----	----	----

Hardware Implementation

Most of these map naturally to rewiring or primitive logic gates.

Sub-bytes, which requires a multiplicative inverse and an affine map, is more complicated.

Non-pipelined initial design -> minimise latency

Process combinatorically, using the Boyar-Peralta method which minimises total gates

This let use maintain our clock frequency.

Hard Inversion
 $GF(2^8)$

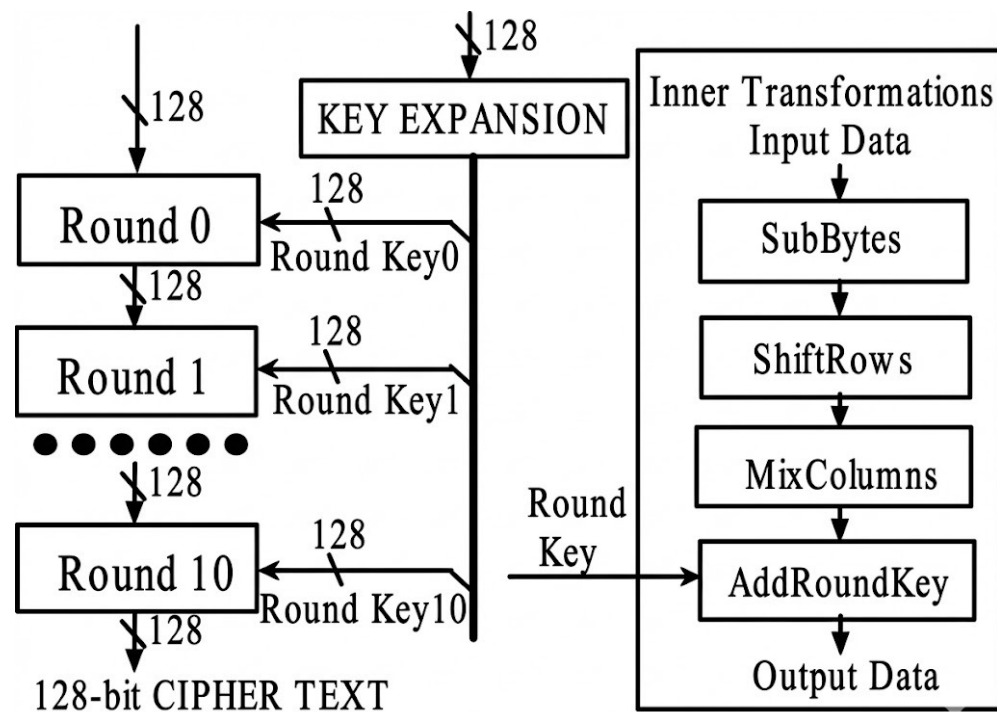


Easy Inversion &
Multiplies
 $GF(2^2)$

$$\frac{1}{a + bi} = \frac{1}{a^2 + b^2} (a - bi)$$

The AES-128 accelerator

- ▶ **Encoder** — standard 10-round AES encryption datapath
- ▶ **Decoder** — full inverse datapath
- ▶ **Key Expansion** — round keys derived algebraically “on-the-fly” — no stored key table, saving silicon.
- ▶ **AES Top** — encoder + decoder + key schedule → the complete accelerator.
- ▶ **Latency** — 11 cycles per 128-bit block through the iterative rounds.

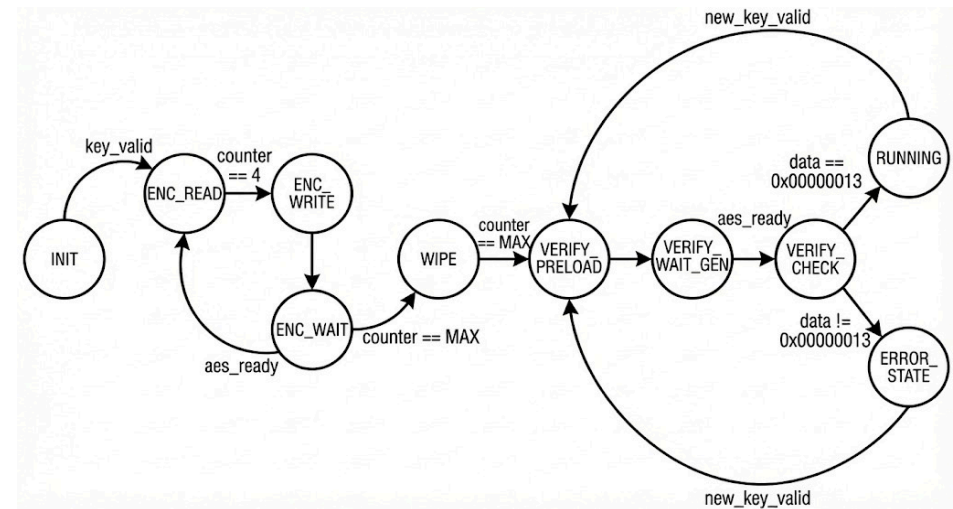


AES-128 round structure

INTEGRATION

System States

- **INIT** - CPU stalled; waits for KEY_VALID from JTAG.
- **ENC_READ / WRITE** - encrypt all plaintext blocks (0–255) into secure SRAM.
- **WIPE** - overwrite all plaintext memory with zeros.
- **VERIFY** - decrypt the reset vector — must match the expected NOP signature.
- **RUNNING** - CPU released; blocks decrypted on demand during fetch.
- **ERROR_STATE** - wrong key → lockdown, PIN_INCORRECT_KEY raised, CPU stalled.

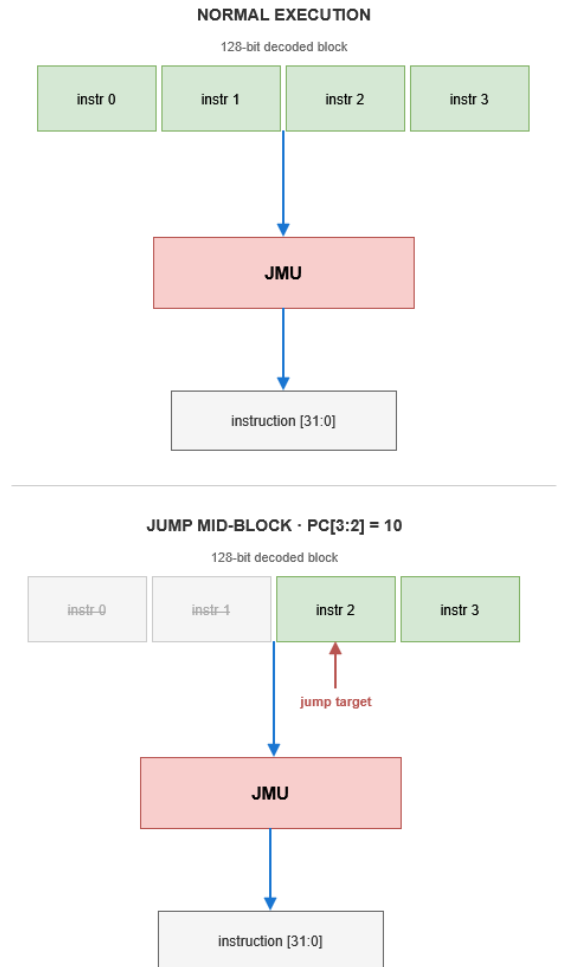


INTEGRATION

Handling jumps across 128-bit blocks

Challenge: each secure word holds 4 instructions, but a taken jump may target any one of them - or a completely different block.

- **decoded_block register** - the decrypted 128-bit word is held in a register.
- **Intra-block jump** - a 4:1 mux selects the target instruction instantly, no re-fetch needed.
- **Inter-block jump** - controller computes the new block index as $PC \gg 4$ (drop byte-offset bits).
- **Stall & flush** - asserts `cpu_stall` to freeze the PC + pipeline regs while the new block decrypts.
- **Correctness** - prevents executing stale / invalid instructions during the fetch.

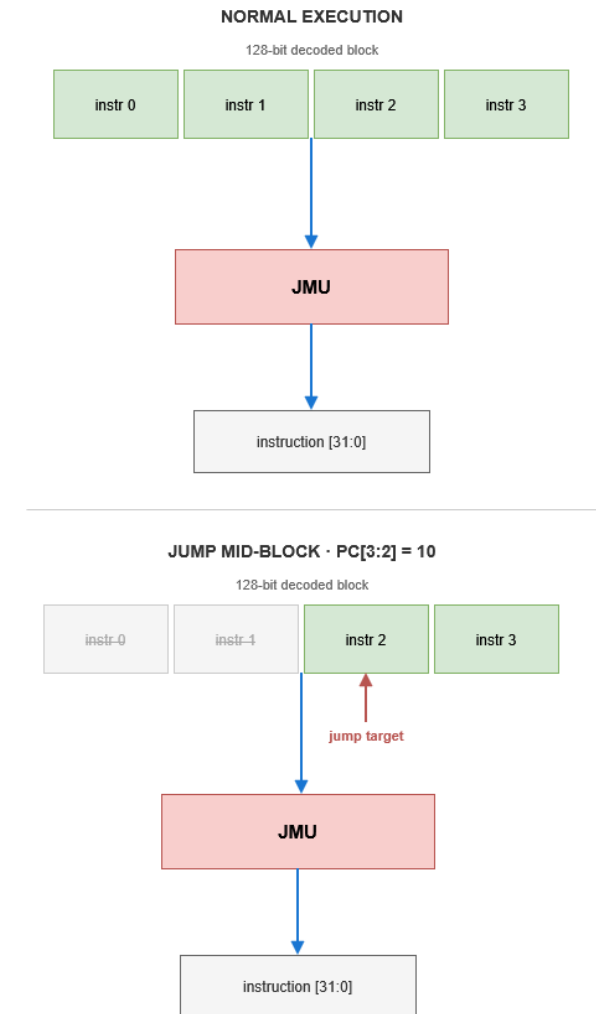


Stretch Goals

Pipelining the Accelerator

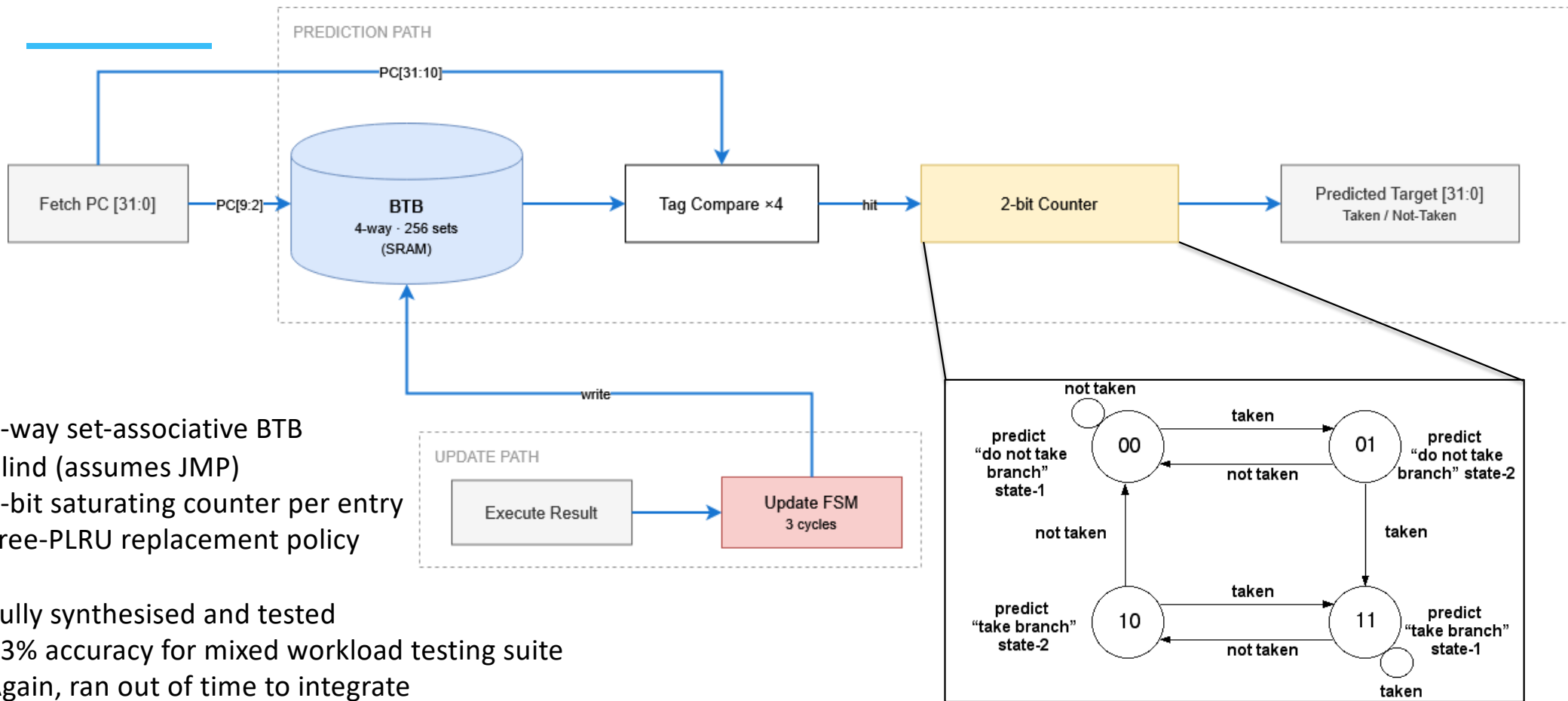
Pipeline and Jump Management

- Decoder split into 3 pipeline stages which run for 4 cycles
- Jumps must now consider whether target is in pipeline
- Encoder pipelining is simpler as no jump management
- Ran out of time to debug the integration with the CPU



Stretch Goals

Branch Predictor



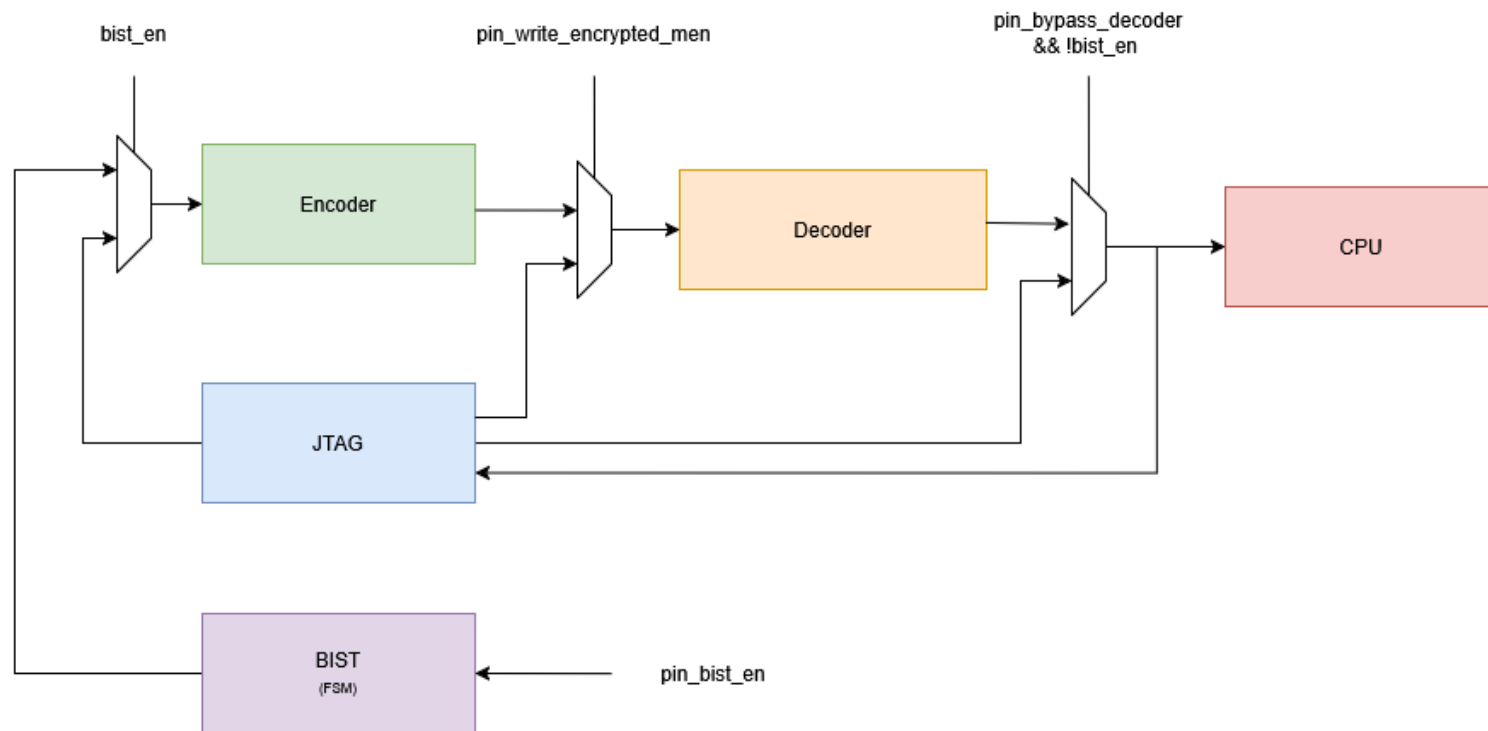
- 4-way set-associative BTB
- Blind (assumes JMP)
- 2-bit saturating counter per entry
- Tree-PLRU replacement policy
- Fully synthesised and tested
- 63% accuracy for mixed workload testing suite
- Again, ran out of time to integrate

Testing Tools and Methods

- **AES Specification**
 - Vectors provided by the AES specification which state the inputs and outputs at each block
- **AES Golden**
 - A python implementation of the AES algorithm, which computes arbitrary outputs and intermediate results.
- **Simulator**
 - cocotb (python) in conjunction with Xcelium and Verilator.
- **VCD Files**
 - The cycle-by-cycle operation of each Module is saved in a VCD file for each test run.
- **Post Synthesis Testing**
 - Gate level simulation (GLS) allowing the timing and functionality to be verified post synthesis.

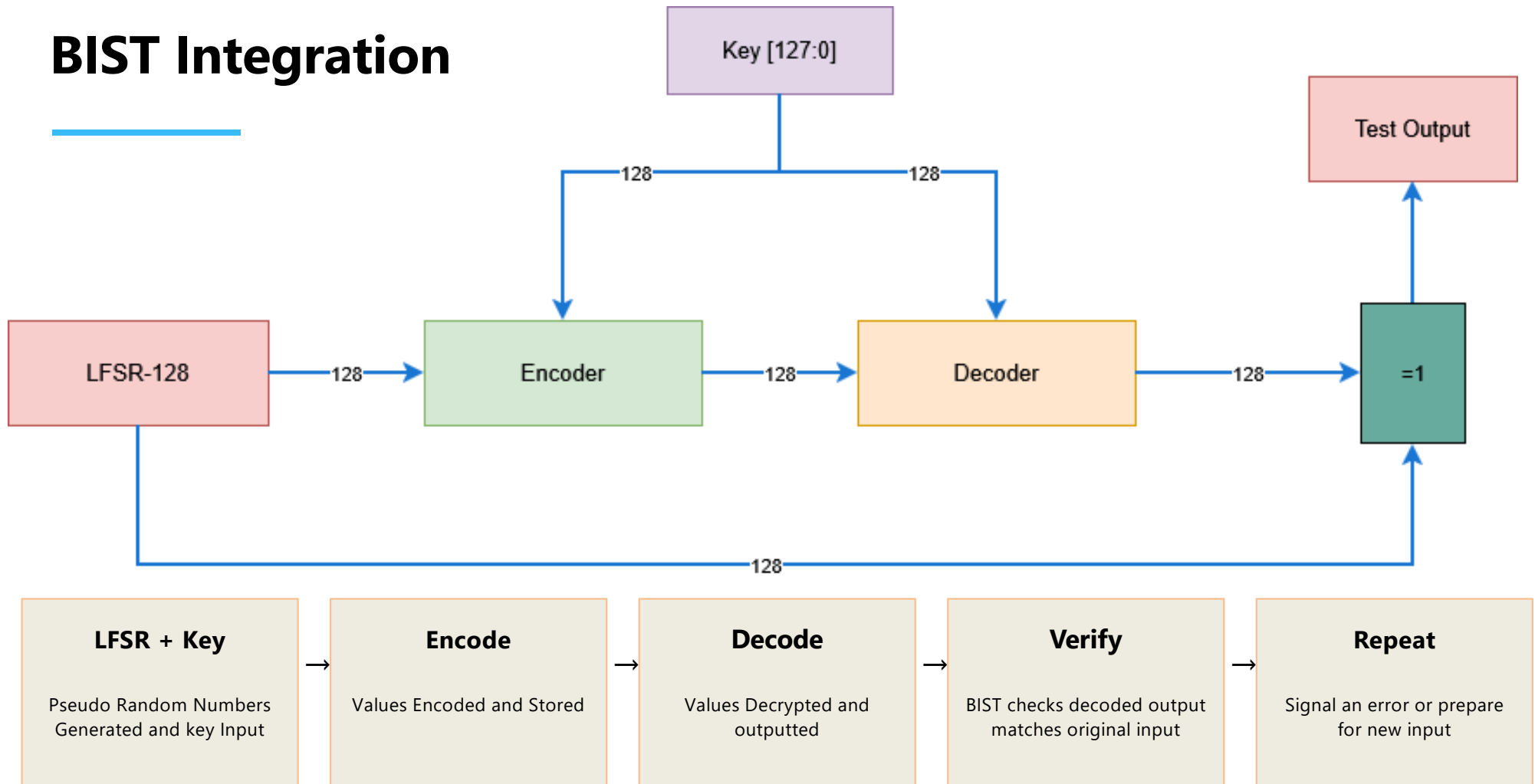
INTEGRATION

Bring Up Plan - Muxes



INTEGRATION

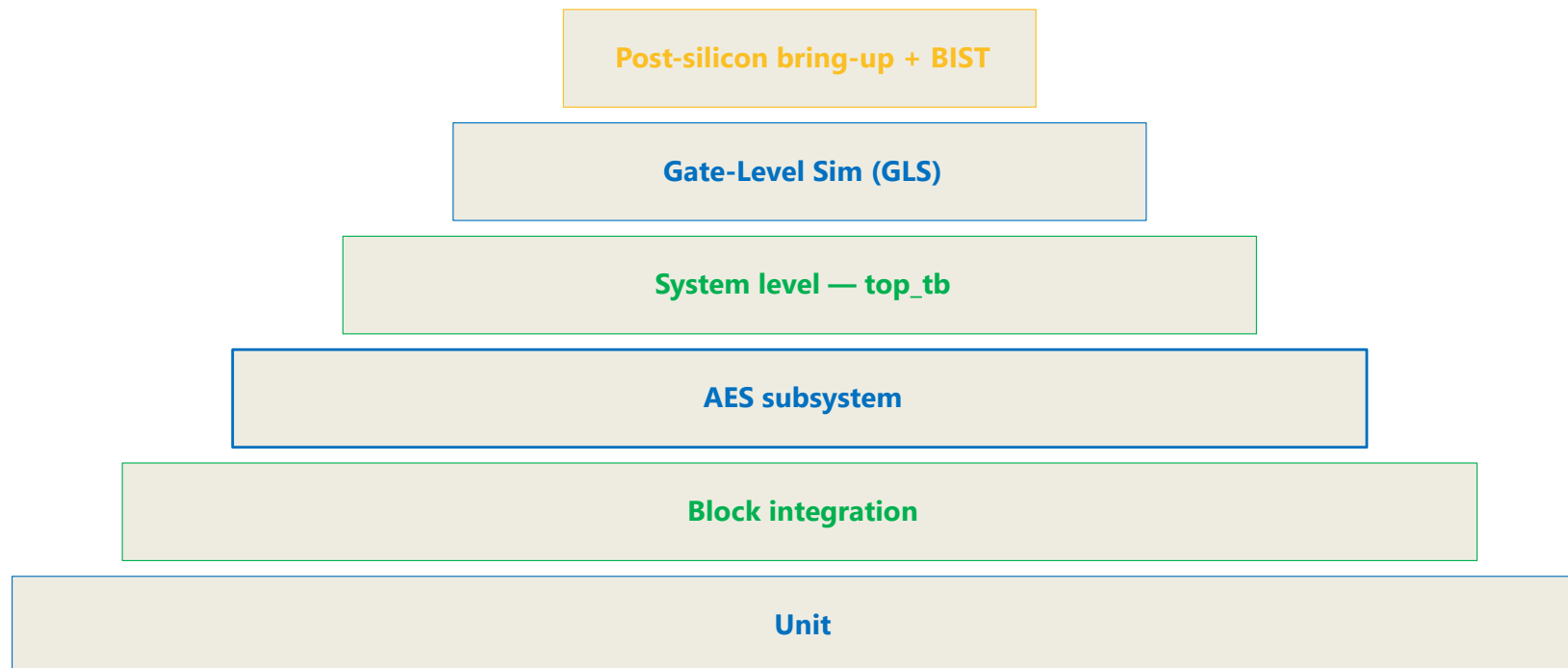
BIST Integration



TEST PLAN

Testing Pyramid

Bottom-up: Test every sub-block, then integrate — **51 cocotb tests + 18 Post Synthesis Tests + post-silicon bring-up plan.**



TEST PLAN

Coverage & methodology

TEST INVENTORY

Level	Tests	Covers
AES unit	12	Each Stage of the AES algorithm
Decoder CPU integ.	6	Tests the JMU and CPU integration
AES subsystem	12	top, jump, stall, pipeline, bypass, JTAG, NOP
BIST	3	controller, success & failure paths
System (GLS)	18	boot, ALU, branches, ld/st, muldiv, UART, timer, stress
Verilator	12	CPU + peripheral smoke: GPIO, JTAG, UART, mem-wipe

Testing flow when designing

1. make test-verilator

CPU & peripheral smoke tests

2. make test-aes

AES unit + subsystem

3. make test-top

full SoC, pre-synthesis

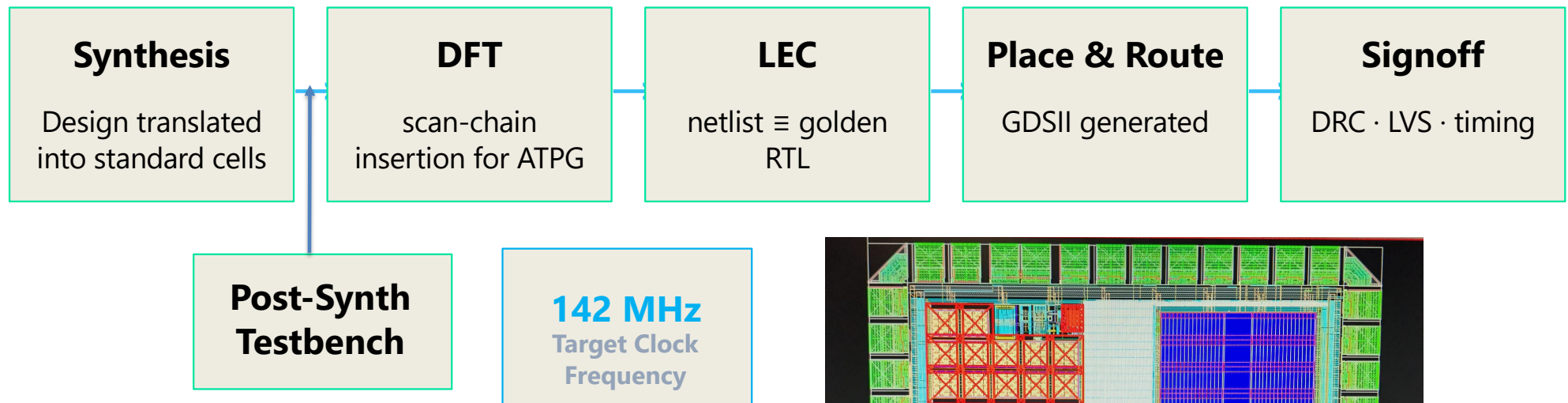
4. make test-top-synth

gate-level (post-synth / GLS)

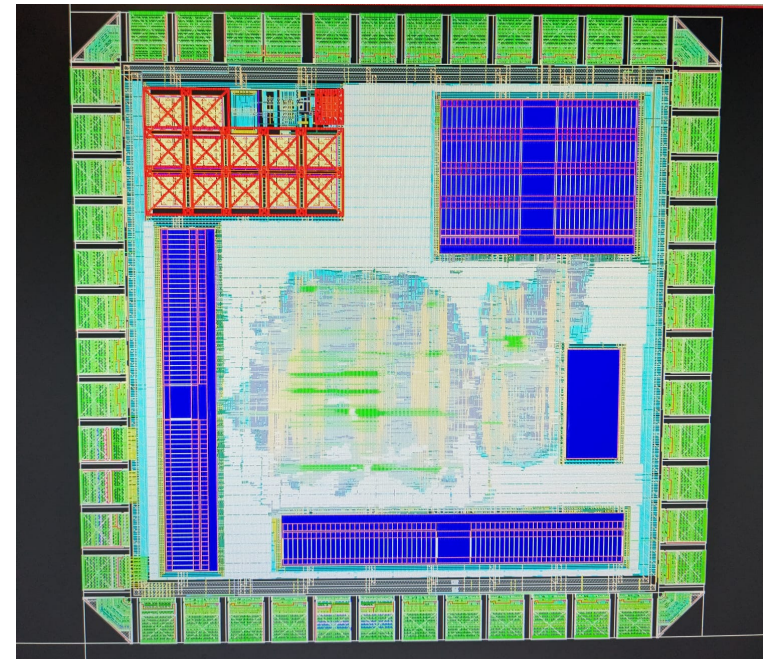
5. make full-test

complete regression

Physical Design



- **Feedback Loop**
- **Memories**
- **Clock --> Standard cells, area**



Key Takeaways

- **Correctly scoped and specified project**
 - Modular, incremental
- **Robust Testing Framework**
 - Unit tests, integration tests, system tests
- **Design For Test Principles**
 - BIST, scan chains, and muxes for controllability and observability
- **Compromised on CLK for faster design process**

Key Takeaways

- **More planning before jumping into RTL**
 - Golden model, interfaces, column vs. row-major
- **Integrate with CPU earlier**
 - Specify interfaces, avoid costly reworks
- **Start physical design earlier**
 - discover bugs faster
- **Reduce scope of goals**
 - Branch-prediction

Results

- **Project Cryptonite successfully modified a RISC-V CPU to protect instruction memory at rest with custom AES-128 cryptography unit running at 142 MHz**
- Passed DRC and LVS (before the deadline ;)
- GDSII approved for Tapeout
- TSMC successfully fabricated our chip w. 65nm process
- Unofficially voted for **best team name**

Thank You!

